SharePoint Hybrid Navigation 202X

Contents

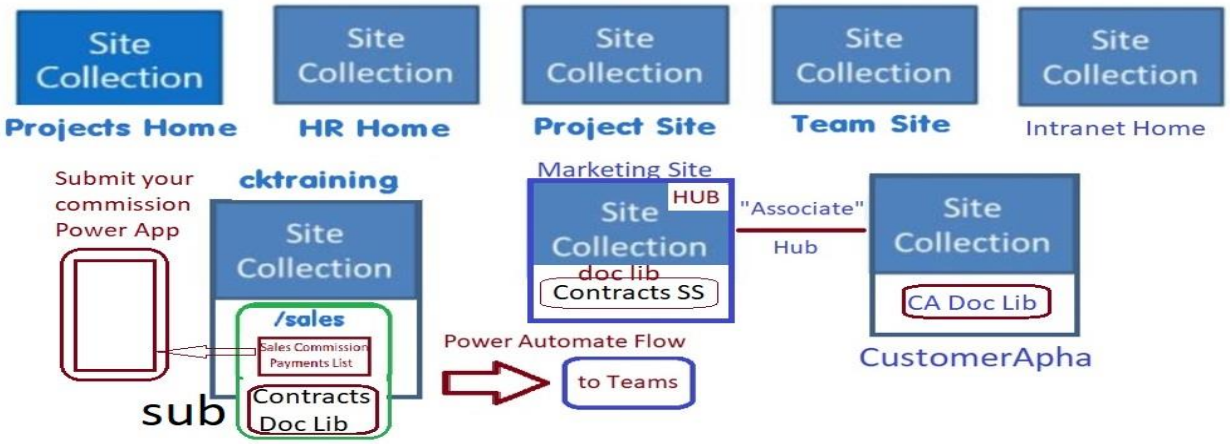
✅ Part 1 — Why basic site collection navigation is so difficult to manage 4
⭐⭐ Solution Pattern: Global Hub → Local Megamenus → Local "Back to Parent" links 5
🚀 Recommended navigation design for YOUR diagram 8
🧭 SharePoint Online Navigation Architecture Blueprint 9
Advanced Navigation 13
Create a mega menu in SharePoint Online (SPO) 15
How to customize the navigation of a communication site by adding a link/label to the top menu 15
Steps to add or edit links/labels in a communication site footer 16
How to identify a team site 18
Navigation Settings Screenshots 18
🧭 Summary: What YOU need to do 24
🧭 SharePoint Navigation Architecture Diagram 25
Generate the SPFx Breadcrumb Extension 28
References 43

You're basically facing **the classic modern SharePoint navigation paradox**:

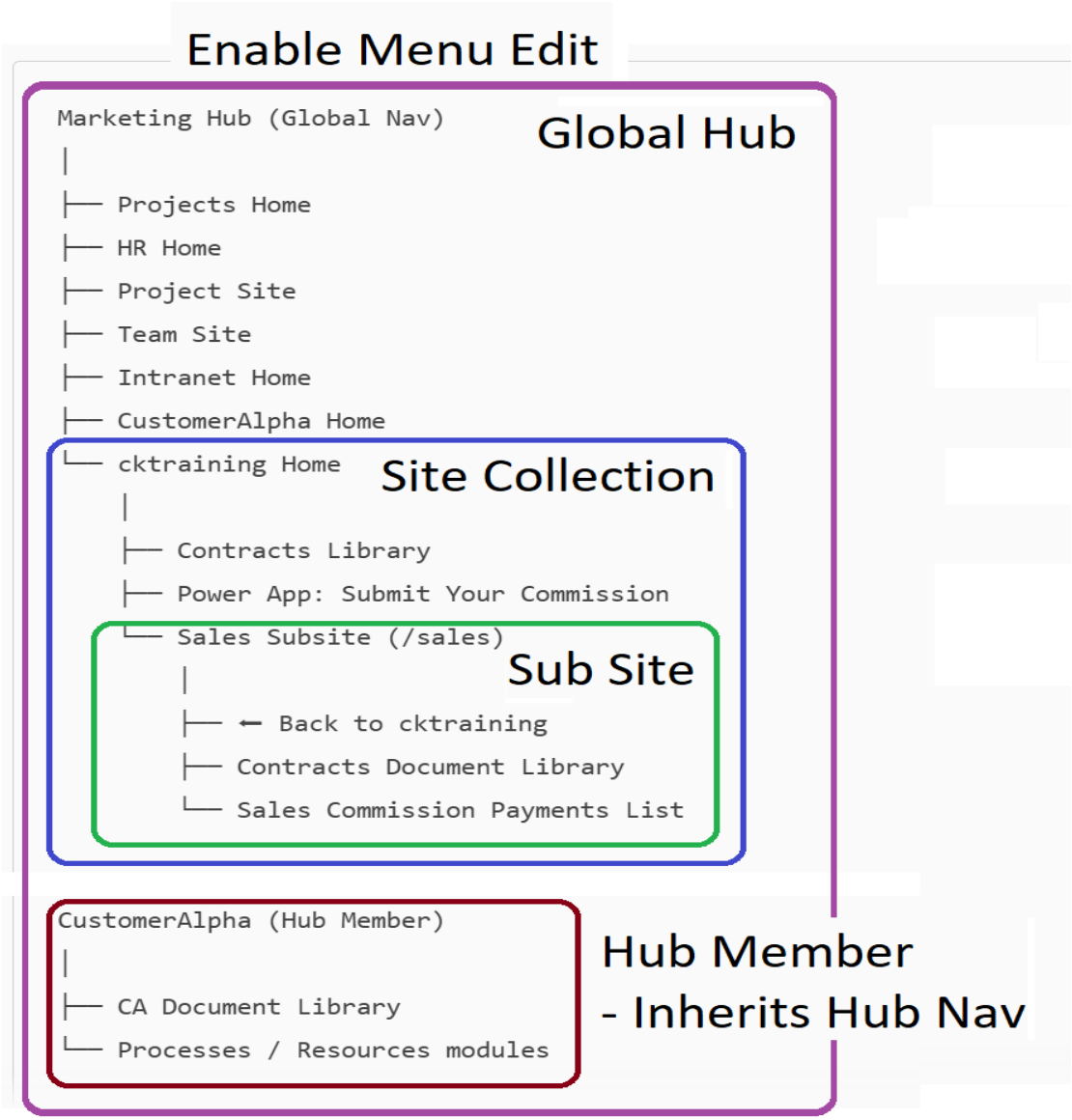
"We have hubs. We have site collections. We have subsites. We have users who need to jump around. Why is it STILL so hard to do simple navigation?"

Add SharePoint Wide Navigation Between Disconnected Site Collections & Sub Sites not in HUB

Copilot Q what is the quick, least maintenance needed way to go up from sub to parent - cross to non hub associated site collection to site/hub associated combo. Why is basic in site collection navigation so difficult to manage?



Your diagram is great — it shows exactly where the problem comes from:
Microsoft designed modern navigation ONLY for flat, hub-based architectures…
…but millions of organizations still have subsites and cross-site dependencies.

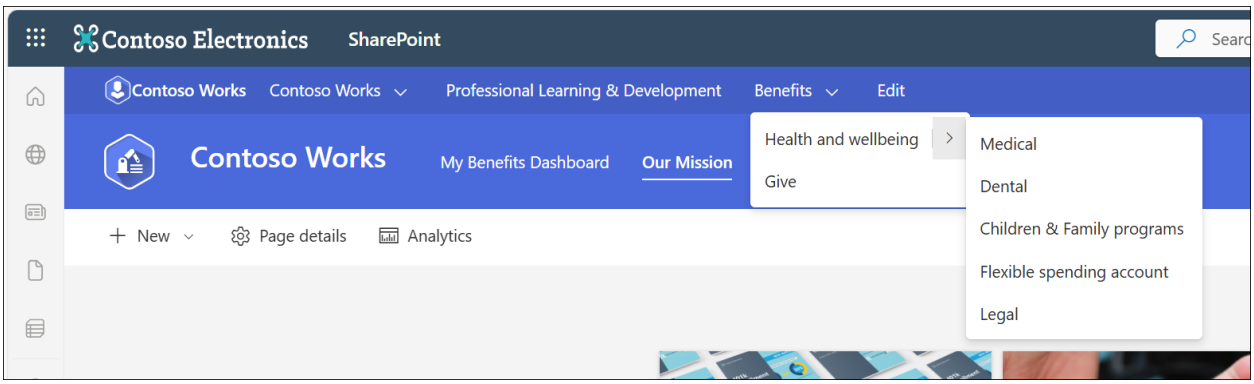


Below is the clearest breakdown of why navigation hybrids are painful *and* the easiest low-maintenance solutions.

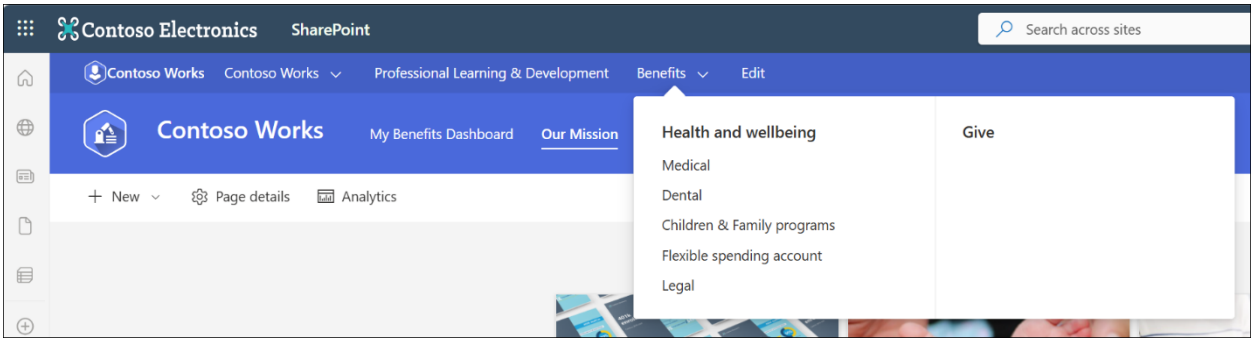
Navigation menus in SharePoint

There are two types of (newer) navigation menu styles for SharePoint sites - cascading and mega menu. Team sites and communication sites support both mega menus and cascading menus. Hubs, whether they are team sites or communication sites, add an additional mega menu to the site.

Cascading menu



Mega menu



✅ Part 1 — Why basic site collection navigation is so difficult to manage

Because **modern SharePoint navigation is intentionally limited to one of three scopes**, and none of them handle subsites + multiple site collections well.

1️⃣ Hubs → Only solve "horizontal" navigation

A hub solves these scenarios:

- Site collection A to site collection B
- Shared top navigation
- Shared theme
- Shared search
- Shared rollups

But **hubs do NOT apply downward to subsites**, which are considered "legacy."

So in your diagram, the Marketing Site (hub) can tie to CustomerAlpha, but the /sales subsite is orphaned from hub features.

2️⃣ Local site collection navigation is isolated

Inside a site collection:

- You can build local top navigation (Megamenu / Cascading)
- You can build local left navigation
- You can add links manually

But:

❌ It doesn't "see" other site collections
❌ It doesn't auto-update
❌ It doesn't inherit the hub nav
❌ It requires manual duplication

This is why you feel the pain.

3️⃣ People still have subsites, and subsites don't participate in modern nav

Subsites:

- Never inherit hub nav
- Never appear in hub rollups
- Can't be associated to hubs
- Have their own separate navigation store

They are **nav dead-ends** unless you manually hack in links.

This is exactly what your /sales subsite is suffering from.

✅ Part 2 — The Quickest, Lowest-Maintenance Navigation Hybrid Pattern

There **is** a pattern organizations use when they are in a hybrid world (some hubs, some standalone site collections, some subsites).

This is the architecture that creates the least admin overhead:

☆☆ Solution Pattern: Global Hub → Local Megamenus → Local "Back to Parent" links

You combine **three navigation layers**, each doing what it's best at:

Layer 1 — Global navigation via ONE hub

Use the *Marketing* site (in your image) as:

✓ The global jumping-off point
✓ A universal "You are here" anchor
✓ A cross-site navigation system

Task: **For sites NOT associated to the hub, you can still add manual links to them in the hub menu.

Purpose: Give users a consistent, always-available way to leave any site and jump anywhere else.

Layer 2 — Site collection–level Megamenu

Inside each site collection:

✓ Build a simple megamenu (3–7 items max)
✓ Include "Home," "Libraries," "Processes," "Resources," etc.

The key rule:

👉 **Do NOT duplicate global navigation locally.**
Only include pages that belong to that site collection.

This keeps maintenance low.

Layer 3 — "Back to Parent" link(s)

For sites not connected by a hub (subsites especially):

Use a simple, consistent convention:

Top-left link or breadcrumb replacement:

⬅ Back to [Parent Site / Home]

Examples:

- From /sales → back to **cktraining home**
- From CustomerAlpha → back to **Marketing Hub**
- From Projects → back to **Projects Home**

This is the only reliable, low-maintenance way for subsites to navigate "up."

Add it using:

- The **local site navigation** (top nav)
- Or a **banner** at the top of key pages
- Or a **custom SPFx navigation extension** (advanced)

🧭 Why is this the least maintenance approach?

Because each layer updates independently without forcing duplication:

Layer	Updates Required	Cost
Hub nav	1 location	**Lowest**
Local megamenu	Per site collection	Moderate
Back-to-parent link	Rarely changes	Lowest

You avoid:

- Duplicating global nav inside local nav
- Maintaining links in 20 places
- Rebuilding navigation every time a site is associated/disassociated from a hub

---

🛑 What NOT to do

These patterns **explode admin cost**:

❌ Trying to force subsites into hub nav
❌ Duplicating the entire hub nav on every site collection
❌ Using 100+ manual quick links as a "navigation replacement"
❌ Dynamically generating nav via JSON every time a site changes
❌ Maintaining 12 hubs instead of 1–3 max

—

🧩 Bonus: Your architecture's tricky point

In your diagram:

- The Marketing site is a hub
- The /sales subsite is NOT a hub member
- CustomerAlpha *is* a hub member

This creates:

🔹 **Sibling navigation = easy** (Marketing → CustomerAlpha)
🔹 **Parent → Subsite = broken** (Marketing Hub → /sales)
🔹 **Subsite → Hub = manual**
🔹 **Subsite → Sibling = impossible without manual links**

This is exactly why it feels harder than it should.

🚀 Recommended navigation design for YOUR diagram

Here is the best minimal-effort layout based on your scenario:

1. Hub navigation (Marketing Hub)

Includes:

- Marketing Home
- CustomerAlpha
- Projects Home
- HR Home
- Team Site
- Intranet

(NOTE: even non-hub sites can be linked in hub nav)

---

2. Local nav for cktraining (the site hosting /sales)

Keep it simple:

- Home
- /sales
- Contracts Library
- Power App link

---

3. Local nav for /sales subsite

Add:

- ⬅ **Back to cktraining**
- Home

- Contracts Library
- Commissions List

---

4. Local nav for CustomerAlpha

Add:

- **⬅ Back to Marketing Hub**
- CustomerAlpha Home
- CA Document Library

This creates a predictable flow:

- **Global → Collections** (hub)
- **Collections → Local content** (megamenu)
- **Local → Parent** (simple back link)

Minimal. Predictable. Maintainable.

SharePoint Online Navigation Architecture Blueprint

Designed for hybrid hubs + standalone site collections + subsites

Maximizes ease-of-use, minimizes maintenance

---

1. Navigation Philosophy for Your Environment

Your environment contains:

- Multiple *standalone* modern site collections
- A *hub site* (Marketing)
- *Subsites* (example: /sales)
- Cross-site processes (Power Automate → Teams)
- Business-specific libraries (Contracts, CustomerAlpha documents)

These generate navigation needs that Microsoft's default model does *not* handle alone.

This blueprint solves the navigation problems in three layers:

🔷 Layer 1: Global Navigation (Hub Level — Marketing Hub)

Purpose: A single, universal "jump anywhere" bar available across all hub-associated sites.

Even sites NOT associated to the hub should still be manually linked here.

Global Hub Menu Structure

Global Navigation (Hub - Marketing Site)

- Home (Marketing)
- Projects Home
- HR Home
- Project Site
- Team Site
- Intranet Home
- CustomerAlpha Home
- cktraining Home

Notes

✓ You only maintain this navigation **in one place** (the hub)
✓ This allows navigation **between any major site collections**
✓ Even standalone sites appear here via manual links

Why this works

Users always have one "escape hatch" to get anywhere important.

🔷 Layer 2: Local Navigation (Each Individual Site Collection)

Purpose: Give users context about *this* site collection — not everything else.

Rule:

Only show what lives inside this site collection. Do not copy the hub menu.

Example — *cktraining* Site Collection

cktraining Navigation (local)

- Home

- Sales Subsite (/sales)

- Contracts Document Library

- Submit Your Commission (Power App)

- Resources

Example — *CustomerAlpha* Site Collection

CustomerAlpha Navigation (local)

- Home

- CA Document Library

- Processes

- Resources

Why this works

Users understand "this is everything *in this site collection*," not the entire company.

🔷 **Layer 3: Subsite Navigation (Legacy Subsites)**

Subsites cannot participate in hub navigation.

So we use the simplest and lowest-maintenance pattern:

Every subsite gets a single, consistent top-level link:

⬅ Back to [Parent Site Name]

Placed in:

- The local top navigation
- OR homepage text banner
- OR page header link

Example — *Sales* Subsite (/sales)

Sales Subsite Navigation

- ⬅ Back to cktraining

- Home

- Contracts Document Library

- Sales Commission Payments List

Why this works

Subsites feel connected without needing hub features (which they can't have anyway).

🧩 How Users Move Through Your Environment

Putting it all together:

From any hub-associated site → anywhere

Use the hub global navigation
(super low maintenance)

Within a site collection

Use the local megamenu
(no duplication, very clean)

From subsite → parent

Use the "Back to [Parent]" link
(works every time, lowest cost)

🔷 OPTIONAL (Recommended If You Want True Breadcrumbs)

If you want subsites to have **real breadcrumbs**, not just a back link:

- Use an SPFx *Breadcrumb Extension*
- Or a JSON navigation override
- Or a custom header placeholder

This adds only **one deploy/update location** → minimal maintenance.

If you'd like, I can generate this SPFx breadcrumb extension for you automatically.

🏗️ Final Deliverable: Your Full Navigation Blueprint Diagram

Here it is in text form — I can also render a graphical diagram if you want:

Marketing Hub (Global Nav)

|

```
├── Projects Home
├── HR Home
├── Project Site
├── Team Site
├── Intranet Home
├── CustomerAlpha Home
└── cktraining Home
  |
  ├── Contracts Library
  ├── Power App: Submit Your Commission
  └── Sales Subsite (/sales)
    |
    ├── ⬅ Back to cktraining
    ├── Contracts Document Library
    └── Sales Commission Payments List
```

```
CustomerAlpha (Hub Member)
|
├── CA Document Library
└── Processes / Resources modules
```

Advanced Navigation

Steps to Use an SPFx Breadcrumb Extension

1. **Environment Setup:** Ensure you have an SPFx development environment set up (Node.js, Gulp, Yeoman).
2. **Create Project:** Run the Yeoman SharePoint Generator (yo @microsoft/sharepoint) and choose Extension -> Application Customizer.
3. **Develop Component:** Use the Top placeholder to render the breadcrumb using React, querying the SharePoint REST API or PnP JS to retrieve navigation nodes.
4. **Test the Extension:** Use the serve.json file in the config folder to test the extension on a live SharePoint site using debug queries.

5. **Package and Deploy:** Build the project (gulp bundle --ship), package it (gulp package-solution --ship), and deploy the .sppkg file to the SharePoint App Catalog.
6. **Install/Activate:** Install the app on the site, and if not using tenant-wide deployment, activate the feature.

Key Considerations

- **Placeholders:** The Application Customizer provides Top and Bottom placeholders for placing the breadcrumb.
- **Best Practices:** Avoid direct DOM manipulation; use SPFx standard placeholders to prevent breaking during updates.
- **PnP Samples:** The PnP SPFx Breadcrumb sample is a robust starting point.

Create a mega menu in SharePoint Online (SPO)

Mega menus enable site navigation to be displayed at-a-glance on communication pages and hub sites. For Local (site) navigation, you must be a site owner and use a communication site or a hub-connected team site. It works best with three levels of hierarchy (Label > Sub-link > Sub-link).

Create a mega menu on communication sites and hub sites - Microsoft Support

Steps to Create a Local Mega Menu:

1. **Enable Navigation Editing:** Go to your site, click **Edit** at the bottom of the left-hand or top navigation menu.
2. **Create Hierarchy:**

1. Click **+** to add a new link or label.
2. To make a link a sub-item, click the ellipses (...) next to the link name and select **Make sub link**.
3. Create up to 3 levels: Header (Level 1) > Link (Level 2) > Sub-link (Level 3).

3. **Configure Mega Menu View:**

1. Go to **Settings** (gear icon) > **Change the look**.
2. Select **Navigation**.
3. Under **Orientation**, choose **Horizontal**.
4. Under **Menu style**, select **Mega menu**.
5. Click **Save**.

Key Tips:

- **Best Use:** Use all three levels of navigation for the best appearance.
- **Limitations:** Mega menus are not supported on classic publishing sites.
- **Labels:** Use Labels (non-clickable text) to organize groups of links.

How to customize the navigation of a communication site by adding a link/label to the top menu

- Go to the SharePoint Communication Site in which you wish to do the customization.
- From the top menu, click on **Edit** placed to the right.
- Move your mouse pointer to the location in the menu where you wish to add a link.
- Click **+**, and the **Add a link** dialog box will appear.
- You can pick from two options: **URL** or **Header**.
- To add a link to the menu, you just need to enter the web page address and give a display name.

- You can also choose **Header** option if you wish to add a label without a hyperlink.
- Go to the page to which you wish to add a link from the menu and copy the URL from the address bar.
- Paste the URL in the **Address bar** and enter the **Display name** that you desire.
- Click **OK** to confirm.
- To save the changes, you can click **Save**.
- You can click on **Cancel** if you want to discard all changes made.
- You could also change the navigation layout and choose either Cascade or Mega Menu.
- To make that change, go to **Change the look** option, and choose **Navigation**.

To perform these actions to customize the navigation you need to have at least the manage lists permission.

Steps to add or edit links/labels in a communication site footer

Before going to the steps to add or edit links/labels in a communication site footer, we would like to enlighten you about some basic facts. The site footer feature is only available on Communication Site Site Pages. You will not find it in Library pages, List Pages, or other pages on your communication site.

- Click on the **Edit** option placed on the right side of your footer.
- You need to have the footer control turned on to find this option.
- Go to the buttons on the left navigation pane and click **+** just above the buttons.
- In the **Add a link** dialog box that appears, enter the desired URL and display name.
- Click **Ok**.
- To edit existing links on the site footer, click the **Edit** option and click on the ellipses that show up near to the link you want to customize.
- From the **Edit**, **Move Up**/**Move Down**, and **Remove** options, choose the right options, and make necessary changes.
- Click on **Save** after making all desired changes.

How to add a link to the navigation bar of a hub site

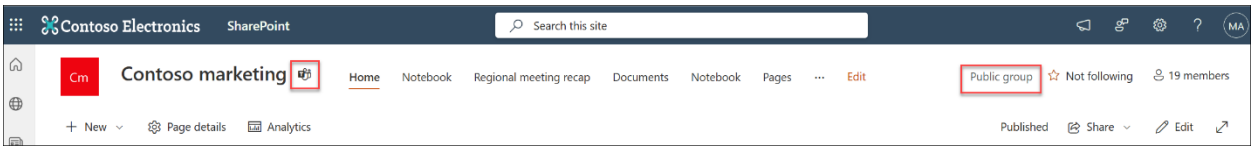
Look for the **Edit** option next to the Hub Site navigation bar. If you do not see that option, it means you do not have permission to make changes to the navigation. At least manage lists permissions are required to perform these steps. The changes you are making to the hub site navigation bar will take up to 2 hours to show up in the associated sites.

- Look for the **Edit** option placed on the right side of the hub site Navigation Bar.

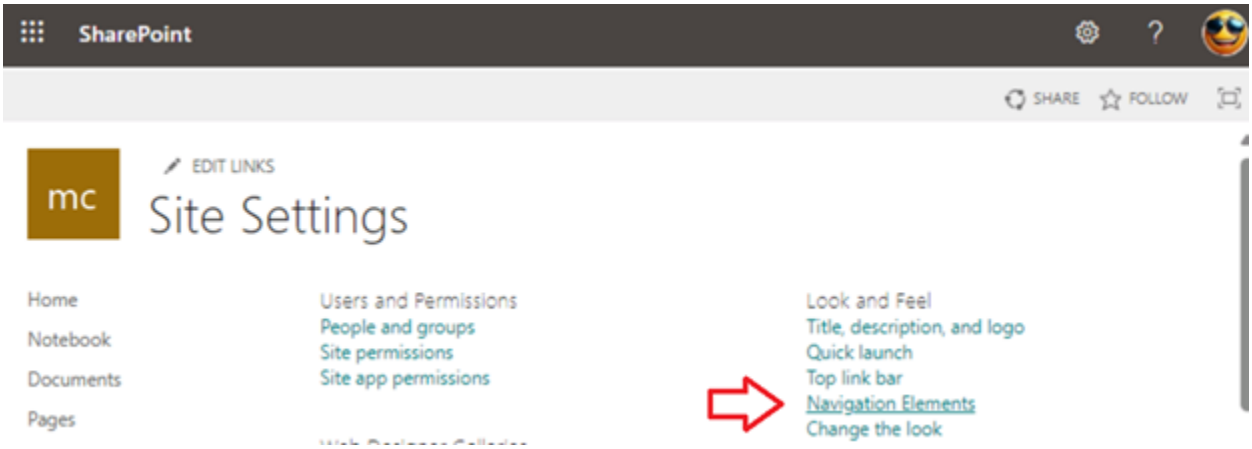
- Click on **Edit** and move your mouse pointer towards the area in the menu where you wish to add a link.
- Click on the **+** icon and the down arrow.
- From the **Add a link** dialog box that appears, you need to fill the **Address** box and **Display name** box.
- To find the correct URL of the web page you wish to add to the navigation bar, go to the page and copy the URL from the address bar.
- Past the URL in the box allotted in the Add a link box.
- Enter an appropriate display name for the box and click Ok.
- The text you are entering in the display name box will be the text that shows up on the menu.
- Click **Save** to complete the process.

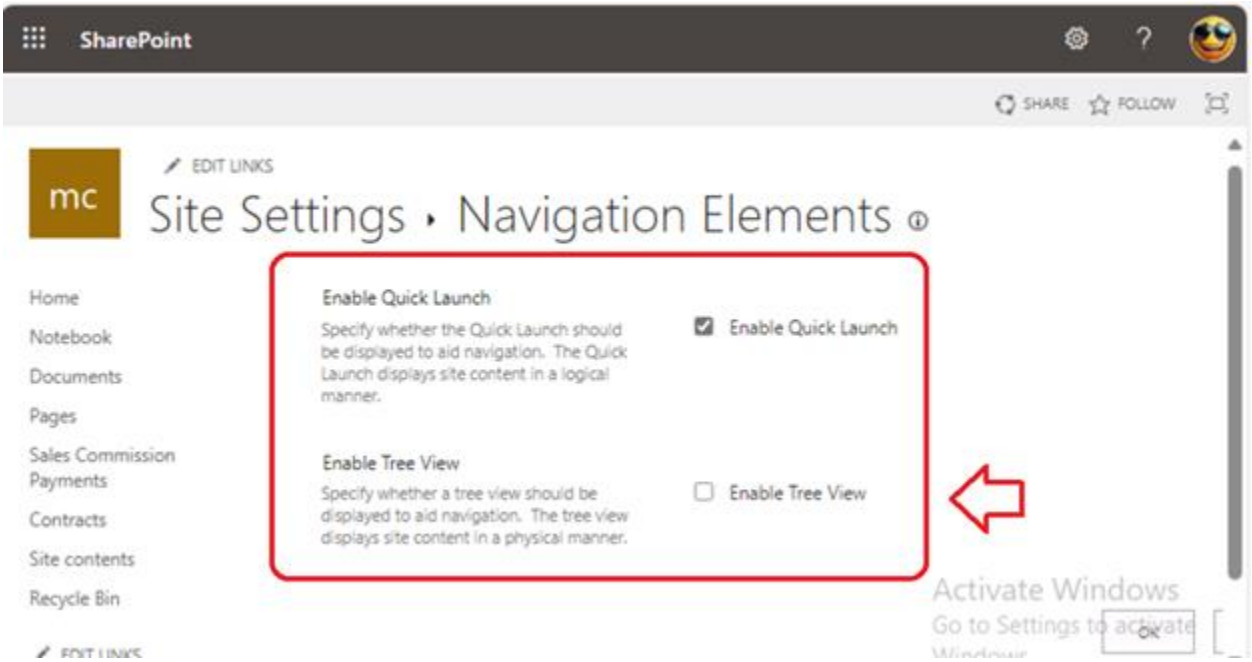
How to identify a team site

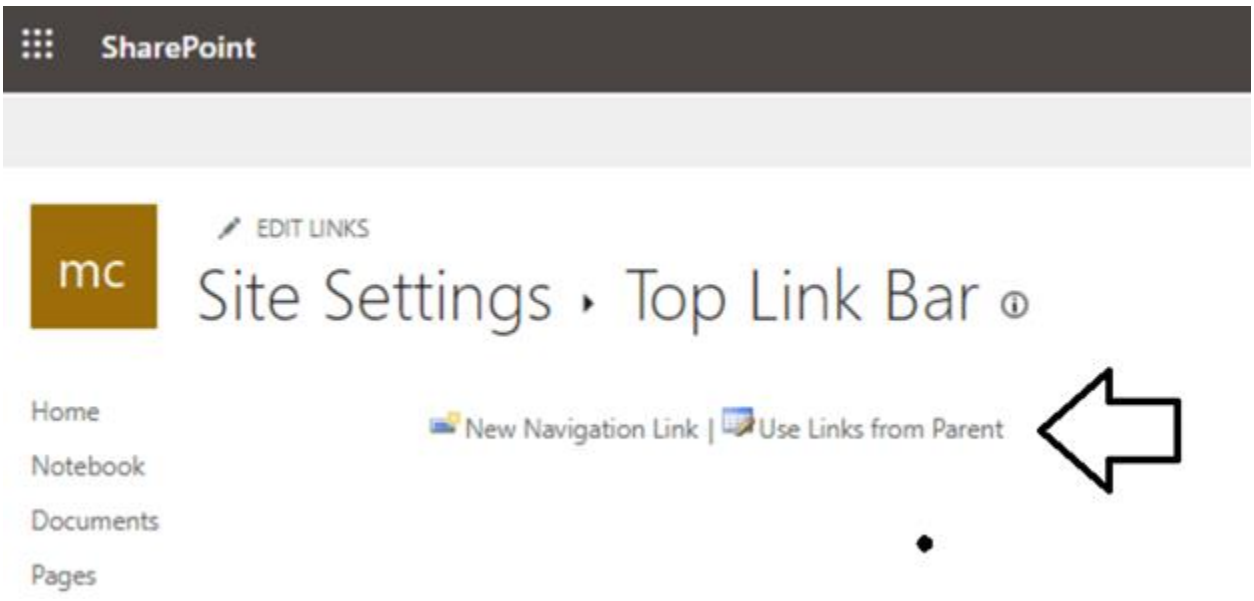
Both team sites and communication sites can have vertical or horizontal navigation bars. However, team sites display a Microsoft Teams channel icon next to the site's title. They also indicate if the site is public or private on the right side of the site header.

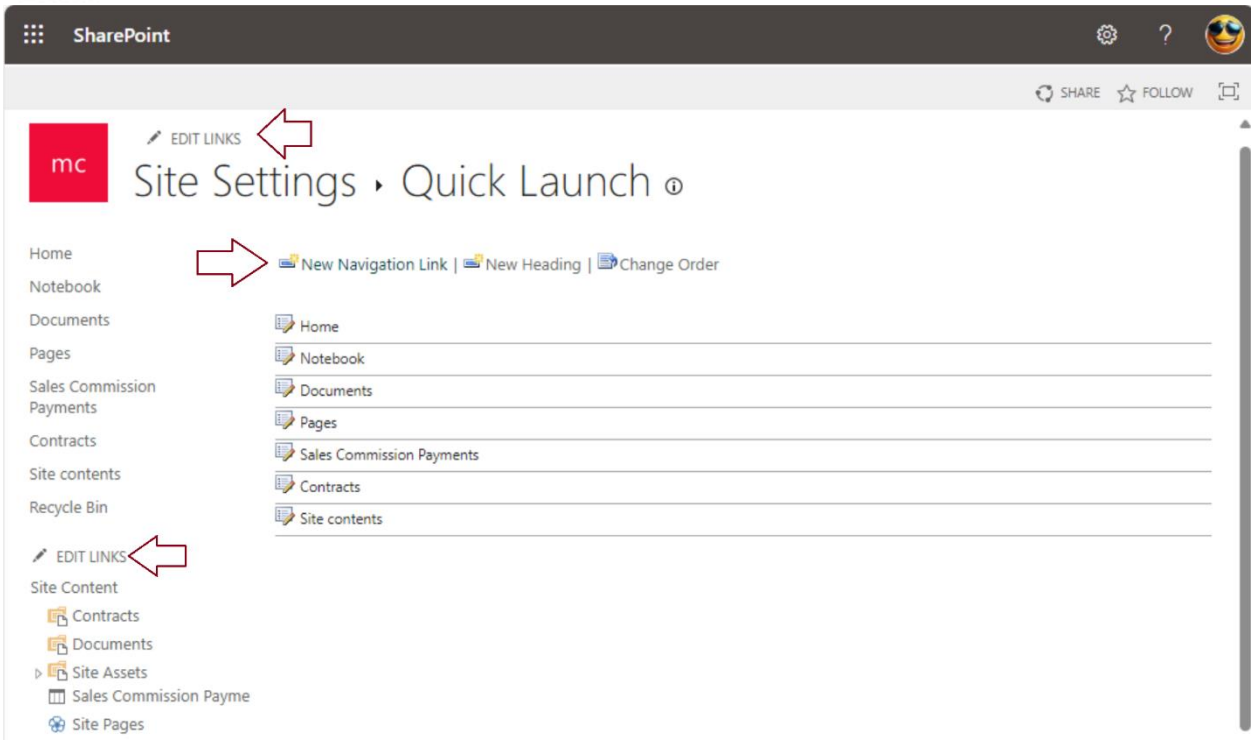


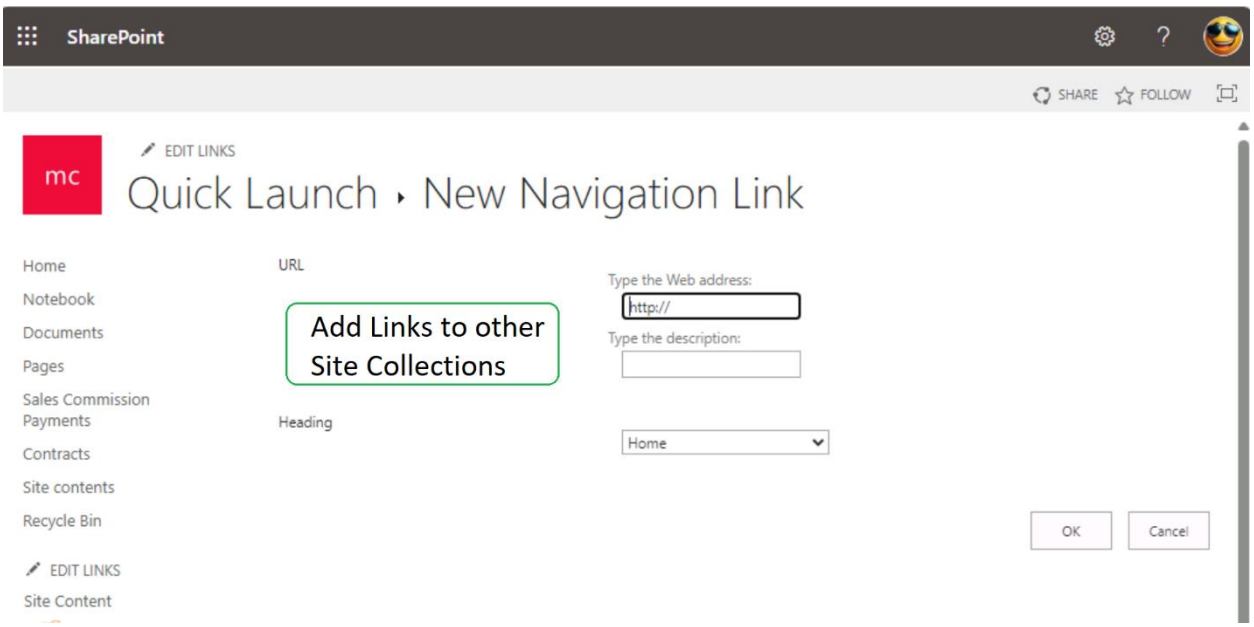
Navigation Settings Screenshots

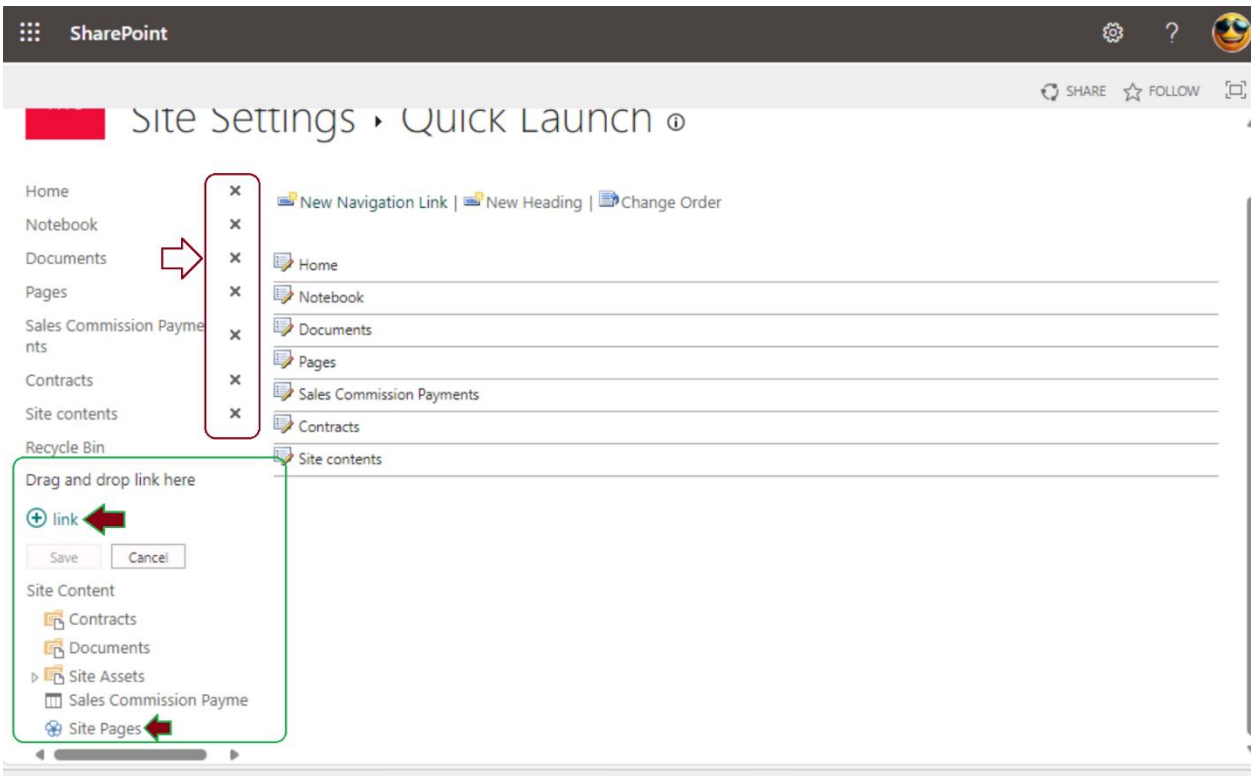





SharePoint
mc
EDIT LINKS
Site Settings › Top Link Bar
Home
Notebook
Documents
Pages
New Navigation Link | Use Links from Parent


SharePoint
SHARE
FOLLOW
mc
EDIT LINKS
Site Settings › Quick Launch
Home
Notebook
Documents
Pages
Sales Commission Payments
Contracts
Site contents
Recycle Bin
EDIT LINKS
Site Content
Contracts
Documents
Site Assets
Sales Commission Payme
Site Pages
New Navigation Link | New Heading | Change Order
Home
Notebook
Documents
Pages
Sales Commission Payments
Contracts
Site contents





is the goal - gather any URLs to manually add for each level - browse to the site and turn on edit for each menu level and add what is needed? Is there a central page in Settings or Site Contents that gets to all edit areas?

Copilot said:

Copilot

Great questions, Tim — and YES, we can break this down cleanly so you understand exactly where navigation is edited, what's centralized vs. NOT centralized, and what URLs you need to gather.

I'll answer your question directly:

✅ **Is there a single central page where you edit ALL navigation (hub, site collection, subsites)?**

No. There is no single central UI that controls all navigation levels in SharePoint Online.

Navigation is edited in **different places**, depending on the navigation *scope*:

Level	Scope	Where you edit it
Hub Navigation	Across all hub-associated sites	**Hub Site → Settings → Navigation / Edit button in top bar**
Site Collection Navigation (modern)	Per site collection	**Site → Edit navigation** (in megamenu/cascading UI)
Subsite Navigation (classic)	Per subsite	**Site Settings → Navigation Elements → Quick Launch / Top Link Bar**

So you are correct:
To fully configure hybrid navigation, **you DO manually browse to each level** and add what you need.

This is why I described the hybrid blueprint — it reduces how many levels you ever need to maintain.

✅ **Your Question: "Is the goal to gather any URLs and manually add them? Do I browse each site and turn on Edit at each level?"**

👉 **Yes. Exactly.**
Let me explain clearly:

🎯 **The Goal in a Hybrid Navigation World**

You are trying to build 3 navigation layers:
Hub Nav → Site Collection Nav → Subsite Nav

Each layer has to be updated **in its own location**, because:

- Microsoft *never built a centralized navigation manager*
- Hubs only control other *site collections*, not subsites
- Modern sites and classic subsites use different navigation engines

So you **collect URLs**, then manually add them to:

1️⃣ Hub Navigation (Global Nav) — Edited in ONE central place

This is the **closest thing** to a global navigation manager.

Where to edit:

- Click **Settings (⚙️)**
- Select **Navigation**
  or
- Click **Edit** in the top navigation bar (your screenshot 1)

Add:

- All other site collections
- Key subsites (optional, not required)

🙌 Only ONE place to maintain.

2️⃣ Site Collection Navigation (Modern Megamenu/Cascading)

These screenshots you posted (Contoso, etc.) apply here.

Where to edit:

- Go to the **root of the site collection**
- Click **Edit** on the top nav

Add:

- Only content *inside the site collection*
  or
- Any important subsites (e.g., /sales)

This is separate from the hub.

3️⃣ Subsite Navigation (Classic)

Your screenshots of:

- Navigation Elements
- Quick Launch

- Top Link Bar
  This is **classic-only** and **subsite-specific**.

Where to edit:

- Subsite → **Site Settings**
- **Navigation Elements**, **Quick Launch**, or **Top Link Bar**

Add:

- Back link ("Back to cktraining")
- Subsite's own lists/libraries
- Maybe a link to the parent hub site

🧠 **Why Microsoft provides no unified navigation control page**

Because SharePoint Online supports:

- **Modern sites** (flat architecture)
- **Hub sites** (cross-collection architecture)
- **Classic sites** (tree architecture)

…and each uses a different navigation provider:

Site Type	Navigation Provider
Modern site collections	SpNavigationStore
Hub navigation	HubSiteData.Navigation
Classic subsites	SPTopNavProvider / SPLeftNavProvider

These are completely different systems internally.

Microsoft never blended them — which is why **you cannot manage all navigation from one screen**.

🧭 Summary: What YOU need to do

✓ Step 1 — Edit Hub Navigation

Visit **ONLY the hub site**
Add links to all important **site collections**.

✓ Step 2 — Edit Each Site Collection Nav

Visit the **root** of each major site collection
Add only internal content + critical subsites.

✓ Step 3 — Edit Only the Necessary Subsites

Visit each subsite
Add:

- Back to parent link
- That subsite's own content

Subsites DON'T need global links.
Leave global nav to the hub.

complete, clean, visual navigation architecture diagram, fully aligned with the hub → site collection → subsite model we discussed.

I'm giving you a **diagrammatic layout using clear ASCII/box drawing**, which you can directly use in documentation, PowerPoint, or governance materials without needing an image generator.

If you prefer an *actual graphical PNG* later (with colors and arrows), I can generate that too.

SharePoint Navigation Architecture Diagram

(Hub → Site Collections → Subsites)

Shows where navigation is edited + what each layer controls

```
┌──────────────────────────────────────────────────────────────────────┐
│                          MARKETING HUB SITE                          │
│                     (GLOBAL NAVIGATION — EDIT HERE)                  │
│                                                                      │
│  Hub Navigation (one place to manage)                                │
│  ├── Home (Marketing)                                                │
│  ├── Projects Home (Site Collection)                                 │
│  ├── HR Home (Site Collection)                                       │
│  ├── Project Site (Site Collection)                                  │
│  ├── Team Site (Site Collection)                                     │
│  ├── Intranet Home (Site Collection)                                 │
│  └── CustomerAlpha Home (Hub Member)                                 │
│                                                                      │
└──────────────────────────────────────────────────────────────────────┘


        ▼ Hub Association (inherits hub navigation)


┌──────────────────────────────────────────────────────────────────────┐
│                      CKTRAINING SITE COLLECTION                      │
│               (LOCAL MEGAMENU — EDIT AT SITE COLLECTION ROOT)        │
│                                                                      │
│  Local Site Navigation (only what lives in this site collection)     │
│  ├── Home                                                            │
│  ├── Contracts Library                                               │
│  ├── Power App: Submit Your Commission                               │
│  └── Sales Subsite (/sales)                                          │
│                                                                      │
└──────────────────────────────────────────────────────────────────────┘
```

```
        ▼ Local navigation only (not hub controlled)


┌──────────────────────────────────────────────────────────────────────────┐
│                        SALES SUBSITE (/sales)                            │
│                   (CLASSIC NAV — EDIT PER SUBSITE)                       │
│                                                                          │
│  Subsite Navigation (classic Quick Launch / Top Link Bar)                │
│  ├── ← Back to cktraining (parent)                                       │
│  ├── Home                                                                │
│  ├── Contracts Document Library                                          │
│  └── Sales Commission Payments List                                      │
│                                                                          │
└──────────────────────────────────────────────────────────────────────────┘



        ▼ Hub Member (inherits hub nav from Marketing Hub)


┌──────────────────────────────────────────────────────────────────────────┐
│                         CUSTOMERALPHA SITE COLLECTION                    │
│                       (HUB NAV + LOCAL NAV EDITED HERE)                  │
│                                                                          │
│  Local Site Navigation (kept minimal)                                    │
│  ├── Home                                                                │
│  ├── CA Document Library                                                 │
│  └── Processes / Resources Modules                                       │
│                                                                          │
└──────────────────────────────────────────────────────────────────────────┘
```

🌟 What this diagram shows at a glance

1️⃣ Hub Navigation — Centralized (best place to manage cross-site nav)

You only need to edit ONE location: the Marketing Hub.

2️⃣ Site Collection Navigation — Per Site Collection

Edited separately on each root site (cktraining, CustomerAlpha, etc.).

3️⃣ Subsite Navigation — Per Subsite (classic)

Edited individually via:

- Quick Launch
- Top Link Bar
- Navigation Elements

Subsites never inherit hub navigation.

Generate the SPFx Breadcrumb Extension

R**eady-to-build SPFx Application Customizer** that injects a modern **breadcrumb / "Back to parent"** bar across your sites and subsites.
It's optimized for your hybrid world (hub + site collections + subsites) and degrades gracefully if hub data isn't available.

What this extension does

- Renders a breadcrumb **below the suite header** on modern pages.
- **Detects hub** association and shows a "Hub Home" crumb when available.
- For subsites, adds **"Back to [Parent]"** and builds a **parent chain** up to the site collection.
- Works on pages, lists, and libraries.
- No custom permissions; uses the current user's context & REST.
- Lightweight (no React dependency required).

---

0) Prereqs (local dev)

- Node.js LTS supported by your SPFx version (Node 16 for SPFx 1.18, Node 18 for SPFx 1.19+).
- Yeoman & SPFx generator:

Shell

```
npm install -g yo @microsoft/generator-sharepoint gulp
```

- A SharePoint App Catalog (tenant or site collection).

If you're unsure which SPFx version to use in your tenant, pick the **latest supported** in your environment and keep the Node version aligned.

---

1) Scaffold the project

Shell

```
yo @microsoft/sharepoint
```

Prompts (recommended):

- **Solution name:** spfx-breadcrumb
- **Target:** SharePoint Online only (latest)
- **Place files:** Use current folder → Yes

- **Deployment option:** Enable tenant-wide deployment → **Yes** (skipFeatureDeployment)
- **Type:** Extension
- **Extension type:** Application Customizer
- **Name:** BreadcrumbApplicationCustomizer
- **Description:** Renders hub/site/subsite breadcrumb

---

2) File structure (key files)

spfx-breadcrumb/ ├– src/ | └ extensions/ | └ breadcrumb/ | ├– BreadcrumbApplicationCustomizer.ts | ├– IBreadcrumbApplicationCustomizerProperties.ts | └ Breadcrumb.module.scss ├– config/serve.json ├– sharepoint/ | └ package-solution.json └ package.json

---

3) Code — **IBreadcrumbApplicationCustomizerProperties.ts**

```ts
export interface IBreadcrumbApplicationCustomizerProperties {
  /** Show hub home as first crumb when hub is available */
  showHub?: boolean;

  /** Maximum number of subsite levels to show (closest ancestors first) */
  maxDepth?: number;

  /** If true, render only "Back to Parent" on subsites */
  backOnly?: boolean;
```

```
  /** Placement: 'Top' (under suite header) or 'Bottom' (above footer) */
  placement?: 'Top' | 'Bottom';
}
```

---

4) Code — Breadcrumb.module.scss

SCSS

```
.breadcrumbHost {
font-family: "Segoe UI", system-ui, -apple-system, Arial, sans-serif;
font-size: 14px;
color: #323130;
background: #faf9f8;
border-bottom: 1px solid #edebe9;
padding: 6px 16px;
}

.wrap {
display: flex;
align-items: center;
flex-wrap: wrap;
gap: 6px;
}

.crumb a {
color: #005a9e;
text-decoration: none;
}

.crumb a:hover { text-decoration: underline; }
```

```
.sep {
color: #8a8886;
}

.backBtn {
display: inline-flex;
align-items: center;
gap: 6px;
color: #005a9e;
text-decoration: none;
padding: 2px 8px;
border-radius: 4px;
background: #ffffff;
border: 1px solid #edebe9;
}

.backBtn:hover {
background: #f3f2f1;
}
``
```

Show more lines

---

5) Code — BreadcrumbApplicationCustomizer.ts

TypeScript

```
import { Log } from '@microsoft/sp-core-library';
import {
BaseApplicationCustomizer,
PlaceholderContent,
PlaceholderName
```

```
} from '@microsoft/sp-application-base';

import { SPHttpClient, SPHttpClientResponse } from '@microsoft/sp-http';

import styles from './Breadcrumb.module.scss';

import { IBreadcrumbApplicationCustomizerProperties } from
'./IBreadcrumbApplicationCustomizerProperties';


const LOG_SOURCE = 'BreadcrumbApplicationCustomizer';


interface IHubSiteData {

title?: string;

url?: string;

}


interface IWebInfo {

Title: string;

Url: string;

ServerRelativeUrl: string;

ParentWeb?: {

Url: string;

ServerRelativeUrl: string;

Title: string;

} | null;

}


export default class BreadcrumbApplicationCustomizer

extends BaseApplicationCustomizer<IBreadcrumbApplicationCustomizerProperties> {


private _topPlaceholder?: PlaceholderContent;

private _bottomPlaceholder?: PlaceholderContent;
```

```
public async onInit(): Promise<void> {
Log.info(LOG_SOURCE, 'Initialized Breadcrumb Application Customizer');

// Defaults
if (this.properties.showHub === undefined) this.properties.showHub = true;
if (!this.properties.maxDepth) this.properties.maxDepth = 3;
if (!this.properties.placement) this.properties.placement = 'Top';

this.context.placeholderProvider.changedEvent.add(this, this._render);
this._render();

return Promise.resolve();
}

private _render = async (): Promise<void> => {
try {
// Create placeholder
if (this.properties.placement === 'Top') {
if (!this._topPlaceholder) {
this._topPlaceholder =
this.context.placeholderProvider.tryCreateContent(PlaceholderName.Top);
}
if (!this._topPlaceholder || !this._topPlaceholder.domElement) return;
await this._mount(this._topPlaceholder.domElement);
} else {
if (!this._bottomPlaceholder) {
this._bottomPlaceholder =
this.context.placeholderProvider.tryCreateContent(PlaceholderName.Bottom);
```

```
}
if (!this._bottomPlaceholder || !this._bottomPlaceholder.domElement) return;
await this._mount(this._bottomPlaceholder.domElement);
}
} catch (e) {
Log.error(LOG_SOURCE, e as any);
}
};

private async _mount(host: HTMLElement): Promise<void> {
const webUrl = this.context.pageContext.web.absoluteUrl;
const siteUrl = this.context.pageContext.site.absoluteUrl;

const isSubsite = (this.context.pageContext.web.serverRelativeUrl.toLowerCase() !==
this.context.pageContext.site.serverRelativeUrl.toLowerCase());

const hub: IHubSiteData | undefined = this.properties.showHub
? await this._tryGetHubData(webUrl)
: undefined;

// Build ancestor chain for subsites
const chain = isSubsite
? await this._buildParentChain(webUrl, siteUrl, this.properties.maxDepth!)
: [];

// Current page (optional): for simplicity, stop at Web level
const crumbs: Array<{ title: string; url: string }> = [];

if (hub && hub.url && hub.title) {
```

```
crumbs.push({ title: hub.title, url: hub.url });
}

// Site collection root
crumbs.push({
title: await this._tryGetWebTitle(siteUrl) ?? 'Site Home',
url: siteUrl
});

// Ancestors (from root down to immediate parent)
chain.reverse().forEach(c => crumbs.push(c));

// Render
host.innerHTML = '';
const wrapper = document.createElement('div');
wrapper.className = styles.breadcrumbHost;

const inner = document.createElement('div');
inner.className = styles.wrap;

if (isSubsite && this.properties.backOnly) {
const parent = chain.length > 0 ? chain[chain.length - 1] : { title: 'Parent', url: siteUrl };
const back = document.createElement('a');
back.className = styles.backBtn;
back.href = parent.url;
back.innerHTML = `
<span aria-hidden="true">←</span>
<span>Back to ${this._escape(parent.title)}</span>
`;
```

```
inner.appendChild(back);
} else {
crumbs.forEach((c, idx) => {
const span = document.createElement('span');
span.className = styles.crumb;
const link = document.createElement('a');
link.href = c.url;
link.textContent = c.title;
span.appendChild(link);
inner.appendChild(span);

if (idx < crumbs.length - 1) {
const sep = document.createElement('span');
sep.className = styles.sep;
sep.textContent = '›';
inner.appendChild(sep);
}
});

// Current web name (not a link)
if (isSubsite) {
const sep = document.createElement('span');
sep.className = styles.sep;
sep.textContent = '›';
inner.appendChild(sep);

const cur = document.createElement('span');
cur.textContent = this.context.pageContext.web.title;
inner.appendChild(cur);
```

```
}
}

wrapper.appendChild(inner);
host.appendChild(wrapper);
}

private _escape(text: string): string {
const div = document.createElement('div');
div.innerText = text;
return div.innerHTML;
}

/** Returns hub title & url if the web is associated with a hub; otherwise undefined */
private async _tryGetHubData(currentWebUrl: string): Promise<IHubSiteData | undefined> {
try {
const resp: SPHttpClientResponse = await this.context.spHttpClient.get(
`${currentWebUrl}/_api/web/HubSiteData`,
SPHttpClient.configurations.v1
);
if (!resp.ok) return undefined;

const data = await resp.json();
// data may be a stringified JSON depending on endpoint behavior
const parsed = typeof data === 'string' ? JSON.parse(data) : data;
const result: IHubSiteData = {
title: parsed?.title || parsed?.Title,
url: parsed?.url || parsed?.SiteUrl || parsed?.siteUrl
};
```

```
if (result.title && result.url) return result;
return undefined;
} catch {
return undefined;
}
}

/** Try to get a web title for a given absolute site/web URL */
private async _tryGetWebTitle(webAbsoluteUrl: string): Promise<string | undefined> {
try {
const resp = await this.context.spHttpClient.get(
`${webAbsoluteUrl}/_api/web?$select=Title`,
SPHttpClient.configurations.v1
);
if (!resp.ok) return undefined;
const j = await resp.json();
return j?.Title;
} catch {
return undefined;
}
}

/** Build parent chain from current web up to site collection root */
private async _buildParentChain(currentWebUrl: string, siteRootUrl: string, maxDepth: number)
: Promise<Array<{ title: string; url: string }>> {

const chain: Array<{ title: string; url: string }> = [];
let cursor = currentWebUrl;
let safety = Math.max(1, Math.min(maxDepth, 10)); // hard cap to avoid loops
```

```
while (safety-- > 0) {
try {
const resp = await this.context.spHttpClient.get(
`${cursor}/_api/web?$select=Title,Url,ServerRelativeUrl,ParentWeb/Url,ParentWeb/ServerRelativeUrl,ParentWeb/Title&$expand=ParentWeb`,
SPHttpClient.configurations.v1
);
if (!resp.ok) break;
const w: IWebInfo = await resp.json();

// If there's no parent or parent is site root, stop
if (!w.ParentWeb || !w.ParentWeb.Url || w.ParentWeb.Url.toLowerCase() === siteRootUrl.toLowerCase()) {
// Add parent if it is not the site root (site root is added separately as a crumb)
if (w.ParentWeb && w.ParentWeb.Url &&
w.ParentWeb.Url.toLowerCase() !== siteRootUrl.toLowerCase()) {
chain.push({ title: w.ParentWeb.Title || 'Parent', url: w.ParentWeb.Url });
}
break;
} else {
chain.push({ title: w.ParentWeb.Title || 'Parent', url: w.ParentWeb.Url });
cursor = w.ParentWeb.Url;
}
} catch {
break;
}
}
return chain;
```

```
}
}
```

Show more lines

---

6) Optional — config/serve.json (local debug target)

JSON

```
{
"$schema": "https://developer.microsoft.com/json-schemas/spfx-build/serve.schema.json",
"serveConfigurations": {
"default": {
"pageUrl": "https://contoso.sharepoint.com/sites/cktraining/SitePages/Home.aspx",
"customActions": {
"BreadcrumbApplicationCustomizer": {
"location": "ClientSideExtension.ApplicationCustomizer",
"properties": {
"showHub": true,
"maxDepth": 3,
"backOnly": false,
"placement": "Top"
}
}
}
}
}
}
```

Show more lines

Run:

Shell

gulp serve

Then append ?loadSPFX=true to your page URL if needed.

---

7) Package configuration — sharepoint/package-solution.json (snippets)

JSON

```
{
"$schema": "https://developer.microsoft.com/json-schemas/spfx-build/package-solution.schema.json",
"solution": {
"name": "spfx-breadcrumb-client-side-solution",
"id": "REPLACE-WITH-YOUR-GUID",
"version": "1.0.0.0",
"includeClientSideAssets": true,
"skipFeatureDeployment": true // enables tenant-wide deployment
},
"paths": {
"zippedPackage": "solution/spfx-breadcrumb.sppkg"
}
}
```

Show more lines

---

8) Build & package

Shell

```
gulp clean
gulp bundle --ship
gulp package-solution --ship
```

Upload **spfx-breadcrumb.sppkg** to your **App Catalog**, and **Deploy**.

---

9) Activate the extension

Option A — Tenant-Wide Extensions (Recommended)

If your tenant has the **Tenant Wide Extensions** list (created automatically when you enable tenant-wide deployment):

1. In the App Catalog site, open **Tenant Wide Extensions** list.
2. Add a new item:
    - **Title:** Breadcrumb
    - **Component Id:** (GUID of the extension from BreadcrumbApplicationCustomizer.manifest.json)
    - **Component Type:** Application Customizer
    - **Location:** ClientSideExtension.ApplicationCustomizer
    - **Properties JSON:**

JSON

```
{
"showHub": true,
"maxDepth": 3,
"backOnly": false,
"placement": "Top"
}
```

- **Scope:** All Sites or target by URL pattern.

This attaches the breadcrumb automatically across selected sites.

Option B — PnP PowerShell (per site)

PowerShell

```
# Connect-PnPOnline -Url https://contoso.sharepoint.com/sites/cktraining -Interactive


$extId = "GUID-FROM-MANIFEST"
Add-PnPCustomAction `
-Title "SPFx Breadcrumb" `
-Location "ClientSideExtension.ApplicationCustomizer" `
-ClientSideComponentId $extId `
-ClientSideComponentProperties '{ "showHub": true, "maxDepth": 3, "backOnly": false, "placement":"Top" }'
```

``

To remove:

PowerShell

Get-PnPCustomAction | Where-Object { $_.Location -eq "ClientSideExtension.ApplicationCustomizer" -and $_.Title -eq "SPFx Breadcrumb" } | Remove-PnPCustomAction -Force

``

---

10) Configuration tips

- **Hybrid best practice** for subsites: set "backOnly": true if you want a simple **Back to Parent** instead of a full crumb chain.
- **Depth control:** if you have deep nesting, adjust "maxDepth".
- **Placement:** set "placement": "Bottom" if your command bar gets crowded.

---

11) Testing scenarios

- **Hub member site:** Should show [Hub] › [Site Collection] › [Current Web].
- **Standalone site (no hub):** Should show [Site Collection] only.
- **Subsite:** Shows parent chain; if backOnly=true, shows **Back** button.
- **Permissions:** Breadcrumb shows only links the user can see; it uses the user's own access.

---

12) Nice-to-haves (next iteration)

- React + Fluent UI version with overflow handling.
- Page title as the last crumb on Site Pages only.
- Audience targeting (hide hub crumb for guests).
- Telemetry via Application Insights.
- Site Script to auto-attach the extension when a site is created.

References

1. Plan and implement SharePoint site navigation - SharePoint in Microsoft 365 | Microsoft Learn
2. **Customize the navigation on your SharePoint site - Microsoft Support
3. Create a mega menu on communication sites and hub sites - Microsoft Support

4. **How To Create a Megamenu in SharePoint Online
5. How to Customize the Navigation of a SharePoint Communication Site and Hub Site
6. Target navigation, news, files, links, and web parts to specific audiences - Microsoft Support
7. Target content to a specific audience on a SharePoint site - Microsoft Support
8. Create and edit a Viva Connections dashboard | Microsoft Learn
9. The Viva Connections dashboard provides fast and easy access to information and job-related tasks. Content on the dashboard can be customized and then targeted to users in specific roles, markets, and job functions. It consists of cards that engage viewers with existing Microsoft Teams apps, Viva apps and services, partner apps, custom solutions using the SharePoint Framework (SPFx) framework, internal links, and external links.
10. What is a SharePoint team site? - Microsoft Support
11. To add links to your page rather than the vertical menu or horizontal menu, see Add a link to a page
12. How to Customize the Navigation of a SharePoint Communication Site and Hub Site